Course Discussions

Solution: Next Greater Element

Problem Statement

Given two integer arrays `nums1` and `nums2`, return an array `answer` such that `answer[i]` is the next greater number for every `nums1[i]` in `nums2`. The next greater element for an element `x` is the first element to the right of `x` that is greater than `x`. If there is no greater number, output `-1` for that number.

The numbers in `nums1` are all present in `nums2` and `nums2` is a permutation of `nums1`.

Examples

1. - Input: `nums1 = [4,2,6]`, `nums2 = [6,2,4,5,3,7]`
   - Output: `[5,4,7]`
   - Explanation: The next greater number for 4 is 5, for 2 is 4, and for 6 is 7 in `nums2`.
2. - Input: `nums1 = [9,7,1]`, `nums2 = [1,7,9,5,4,3]`
   - Output: `[-1,9,7]`
   - Explanation: The next greater number for 9 does not exist, for 7 is 9, and for 1 is 7 in `nums2`.
3. - Input: `nums1 = [5,12,3]`, `nums2 = [12,3,5,4,10,15]`
   - Output: `[10,15,5]`
   - Explanation: The next greater number for 5 is 10, for 12 is 15, and for 3 is 4 in `nums2`.

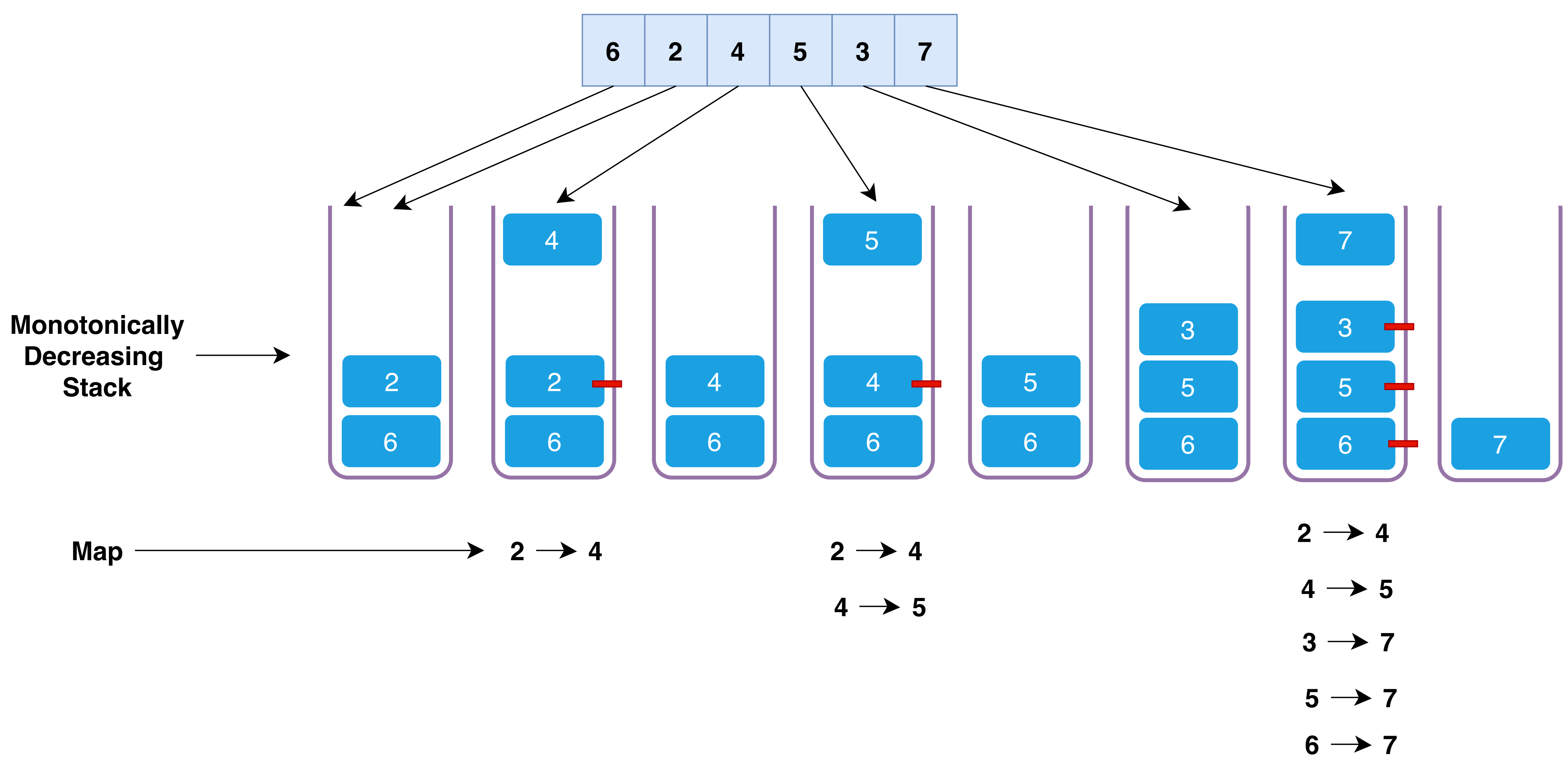
Solution

This problem requires us to find the next greater element for each element in `nums1` by searching `nums2`. The output array contains either the next greater element or -1 if no such element exists.

This problem can be solved using a Monotonic Stack and Hashmap. The Monotonic Stack helps to find the next greater element for each element in `nums2`. The Hashmap then maps each element to its next greater element.

Algorithm Walkthrough

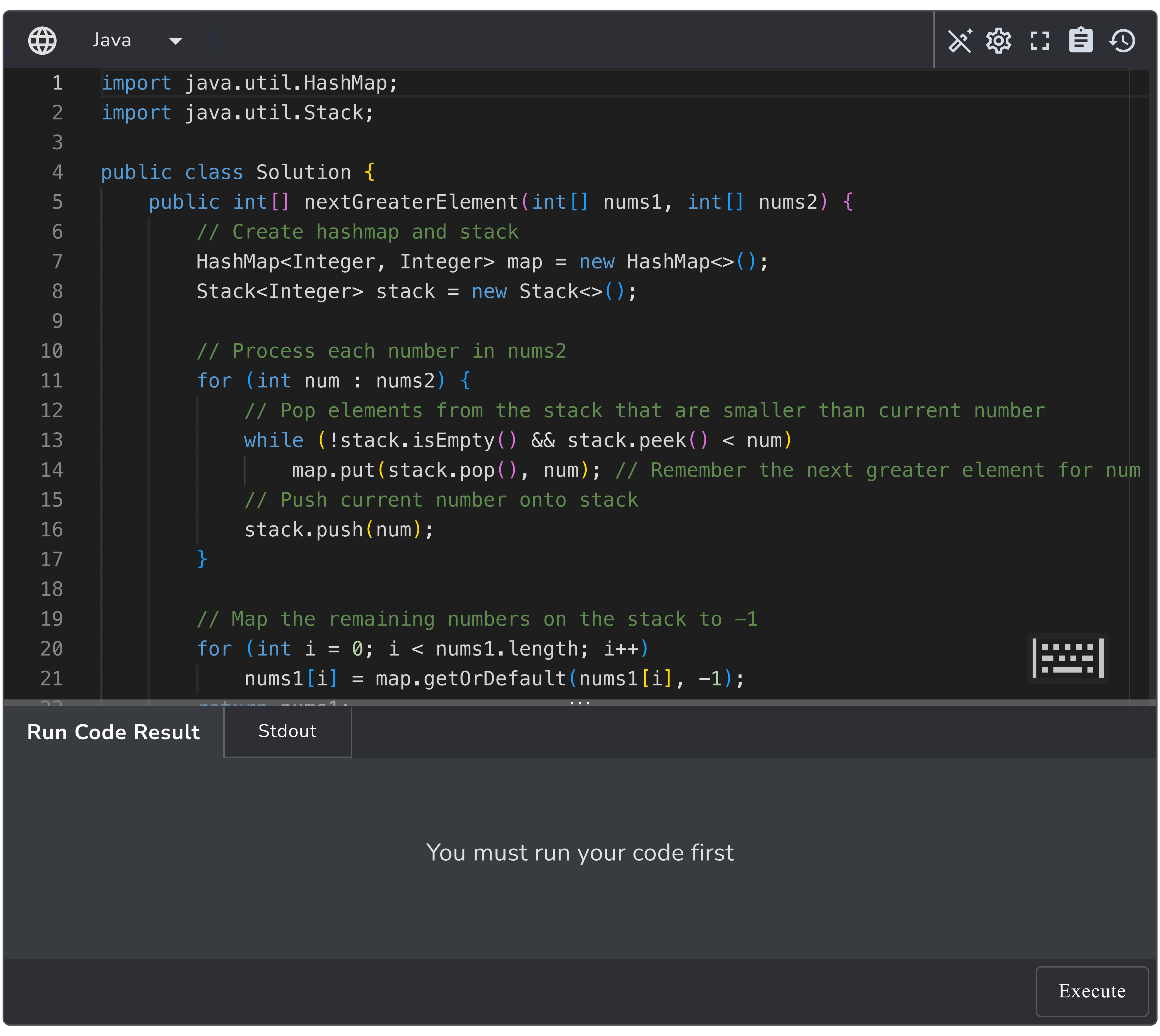
1. Initialize an empty stack and an empty hashmap.
2. Loop through the `nums2` array from left to right.
3. For each number in `nums2`, while the stack is not empty and the current number is greater than the top element of the stack:
   - Pop the element from the stack and add an entry to the hashmap with the popped element as the key and the current number as the value.
4. Push the current number onto the stack.
5. After the loop, any numbers remaining on the stack do not have a next greater element, so for each of these numbers, add an entry to the hashmap with the number as the key and `-1` as the value.
6. Finally, create an output array by mapping each number in `nums1` to its corresponding value in the hashmap.



Next Greater Element through Monotonically Decreasing Stack

Code

Here is the code for this algorithm:



```java
import java.util.HashMap;
import java.util.Stack;

public class Solution {
    public int[] nextGreaterElement(int[] nums1, int[] nums2) {
        // Create hashmap and stack
        HashMap<Integer, Integer> map = new HashMap<>();
        Stack<Integer> stack = new Stack<>();

        // Process each number in nums2
        for (int num : nums2) {
            // Pop elements from the stack that are smaller than current number
            while (!stack.isEmpty() && stack.peek() < num)
                map.put(stack.pop(), num); // Remember the next greater element for num
            // Push current number onto stack
            stack.push(num);
        }

        // Map the remaining numbers on the stack to -1
        for (int i = 0; i < nums1.length; i++)
            nums1[i] = map.getOrDefault(nums1[i], -1);
```

You must run your code first

Time and Space Complexity

The time complexity of this algorithm is $O(N)$, where N is the length of `nums2`, since each element is pushed and popped from the stack exactly once. The space complexity is $O(N)$ as well, due to the additional space needed for the stack and the hashmap.

← Previous
Next Greater Element (Easy)

Next →
Daily Temperatures (Easy)

☐ Mark as Completed